

# What changes from Red Shell

The errors in this deck are Red Shell's, and so are their standards: they carry over. These four rules moved, and breaking any one of them now fails a task outright.

## 1 One prompt, one answer

The task type is always **single turn**: Model A gets the initial prompt and no follow ups. A Model A conversation with more than one user turn **fails the task**.

- **No milestones, no deferred files, no revision turns.** Every requirement goes into the one prompt.
- Leg B still runs the same prompt word for word, steered with **intent level hints**; a hint that supplies a value the model then uses fails the task.

Guidelines · 3. Task Runner - Leg A · QC spec · Task Type - Single-Turn Structure

## 2 Model A has to fail at least 30%

Genuine failures worth **at least 30% of the final rubric score**, with real impact on the deliverable, never forced or artificially scored. **50% or more is preferred**.

**Model A scores 70% or less**

Its failures clear the 30% floor

**Above 70%**

Fails as a Trivial Task

Guidelines · Hard Client Requirements for Every Task · QC spec · Rubric Criteria - All Criteria Scoring

## 3 Two automatic fails in the rubric

**Completion · 80% or more**

What was delivered: the artifact, the state change, the final message

**Process ·**

**20% max**

How it got there

- ✗ **Process over the cap.** Count the objective block and round down: 12 criteria allow 2. Zero is preferred.
- ✗ **Any existence check.** A criterion that only verifies a file, section, column or record is there.

Guidelines · 5.1 Rubric Fundamentals · 5.6 How the Client Defines Rubric Issue Severity

## 4 Every assigned parameter is binding

All **fourteen** parameters, from the scenario and the assigned tools to the Execution target, are followed exactly. **Omitting, substituting or drifting from one makes the task invalid and subject to automatic rejection.**

- Claiming the universe cannot support the scenario when it can **fails the task automatically**: explore before you deviate.
- **11 use cases and 68 subcategories** replace the old 6 and 16.

Guidelines · Hard Client Requirements for Every Task · 1.1 Task Parameters & Execution Rules

**ALSO NEW** At least 3 multimodal inputs, most images needing real visual interpretation · **Every literal** in a criterion copied from its source, subjective block included

# Common Errors

The mistakes that cost people their tasks in Red Shell, restated for the single turn project, and what to do instead.

**Every error in this deck cites its source.** Each one comes with a direct reference to the exact section of [the Green Shell Guidelines](#) or [the QC spec](#) that sets the correct standard, so you can check it yourself. **Each example number is a link** to the real Red Shell task where it happened.

**Every example in this deck comes from Red Shell**, and each one illustrates an issue that must not be repeated in Green Shell: the same quality standards and expectations still apply. Many of these errors break the most basic standards of the project, and correcting them usually takes very little extra effort.

**These are not minor issues.** Repeating these mistakes can directly affect your access to tasks and your ability to remain on the project.

HOW TO USE THIS DECK

# Fix order is build order

The seven steps of a Green Shell task, and the sections whose mistakes are made at each. The earlier the step, the more a fix costs.

| STEP | WHAT YOU ARE DOING                                                                                                                                | SECTIONS WHOSE MISTAKES ARE MADE HERE                                                                                                                                                                 |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1-2  | <b>Plan and set up</b> · universe, inputs and the prompt <b>BEFORE YOU RUN</b><br>Guidelines · 1. Planning the Agent Task · 2. Setting Up the Run | 1 The Foundations Every Task Must Get Right<br>2 What you may not tell Model A or Model B · the prompt and the input folder both legs share                                                           |
| 3-4  | <b>Run Model A</b> · one prompt, no follow ups <b>LEG A · MODEL A</b><br>Guidelines · 3. Task Runner - Leg A                                      | 6 Closing the task · the preferred run                                                                                                                                                                |
| 5    | <b>Write the objective rubric, rate Model A</b> <b>LEG A · MODEL A</b><br>Guidelines · 5. Objective Rubrics                                       | 3 A rubric that separates two runs<br>4 Criteria a correct run can pass                                                                                                                               |
| 6    | <b>Reach the golden</b> · hint Model B until it passes <b>LEG B · MODEL B</b><br>Guidelines · 6. Golden Solution - Leg B                          | 2 What you may not tell Model A or Model B · every hint you send Model B<br>4 Criteria a correct run can pass · the golden passes every objective criterion<br>6 Closing the task · the golden itself |
| 7    | <b>Write the subjective block, rate Model A and the golden</b> <b>LEG B · MODEL B</b><br>Guidelines · 7. Subjective Rubric Block                  | 5 The subjective block                                                                                                                                                                                |
|      | <b>Before you submit</b>                                                                                                                          | 6 Closing the task · the close out                                                                                                                                                                    |

# The Foundations Every Task Must Get Right

Two decisions made while the task is still on paper. Each one is cheap to check and expensive to get wrong.

## The complexity bar

The client sets a minimum floor for each deliverable type and ranks the types by priority. Clear it while you are still designing the scenario: the artifact the prompt defines shapes the inputs, the workflow and the rubric that follow.

**Explainer video, P0** · several scenes driven by the task's data, not one static frame with narration

**Interactive HTML, P0** · controls that respond, state that changes; a styled static page does not qualify

**Dashboard, P0** · several linked views over data the model extracted itself

**Presentation, P1** · 10 to 15 slides carrying a structured narrative

**Designed PDF, P1** · multi page, with a layout that was designed

**CSV or structured data, P1** · multi row, derived from the media, in the target platform's schema

**Document or report, P2** · sections, tables, and figures the text refers to

**Why** Raising complexity is one of the client's clearest and most consistent requirements. Only the prompt and the inputs reach the agent, so a deliverable the prompt does not ask for cannot be graded.

SOURCE

Guidelines · Complexity Bar

Guidelines · 1.3 Defining the Draft History

SEE RED SHELL EXAMPLE 1 ↗

## The universe has to hold what the task needs

The loadout decides what the task can be, so explore the assigned universe before you draft anything. Undocumented does not mean infeasible: test the workflow before ruling it out.

**Confirm what is loaded** · which servers and data sources the session really has

**Ask the in universe agent** · it tells you what it can reach and what it cannot

**Query the data** · SQL shows the tables, relationships and edge cases the visualizers never surface

**Anchor the dates** · universe calendars hold fixed dates, so write the range you saw into the prompt

**Confirm there is enough material** · the universe and the inputs sustain the prompt and every other parameter

**Why** Data the server does not hold, asked for anyway, is your defect, not a model failure. QC fails a task whose primary request the available tools cannot answer, and a drift later shown to be an exploration gap fails too.

SOURCE

Guidelines · 1.2.1 Start With the Universe

Guidelines · 4. Valid vs. Invalid Model Failures

QC spec · Trajectory - Feasibility With Tools

SEE RED SHELL EXAMPLE 2 ↗

# What you may not tell Model A or Model B

Leg A defines the task: one prompt to Model A, with no follow ups. Leg B is the golden run: Model B gets the same prompt, word for word, then the hints you send it. The no leak rule and the input folder are the same in both legs.

## Nothing you type gives the answer away

Not the prompt, not a Leg B hint: nothing you type may give Model A or Model B a value, count, position or conclusion it should derive from the inputs.

**The prompt** · keeps thresholds, rules and formats in the input files, where each model must find them

**The input folder** · filenames, manifests and helper documents never reveal the expected answer

**Why** Once the value is in a message, the attachment is decoration: no criterion that depends on reading it can separate a run that opened the file from one that did not.

SOURCE

Guidelines · 1.2.3 Create the Prompt  
Guidelines · 1.2.2 Select the Multimodal Inputs  
Guidelines · 6.1 What Changes from Leg A

SEE RED SHELL EXAMPLES 3 AND 5

## One prompt, the same in both legs

Leg A is one prompt and one answer: Model A gets no follow ups, so there is no turn to correct it in. Leg B sends Model B the same prompt word for word, with the same intent and the same persona throughout.

**Leg A** · a Model A conversation with more than one user turn fails the task

**The prompt** · carries the whole request and names every output file verbatim

**Leg B** · hints only, at intent level, until Model B reaches the golden

**Why** Both legs run the same prompt word for word, so any difference between the two runs comes from the models, never from what you told them.

SOURCE

Guidelines · 3. Task Runner - Leg A  
QC spec · Task Type - Single-Turn Structure  
Guidelines · 6.2 Steering the Run & Hinting

## A Leg B hint points at the area, never at the answer

Say what is still missing, from the user's point of view, and point at the area or the source. "A is wrong" is fine; "A is wrong, do B instead" fails.

**Fails the task** · a hint supplies a value the model then uses without going back to the input

**Costs points** · over steering, such as naming the exact file to open

**Why** A golden that meets a criterion because Model B was told the answer no longer shows the reasoning the criterion grades.

SOURCE

QC spec · Golden Solution - Hint Leak (Leg B)  
Guidelines · 6.2 Steering the Run & Hinting

SEE RED SHELL EXAMPLES 4 AND 5

# A rubric that separates two runs

A rubric exists to tell a strong run from a weak one. Five standards keep it doing that, and a set can break any of them while still looking complete.

## Negatives catch the traps

Positives are the default. Negatives are the exception: around 25% of the block, generally below 30%, each a specific, plausible failure the scenario or trajectory exposes, never one added for balance.

**Why** The negative slots are for attractive wrong paths the model has to resist. Present on a negative means the run did the bad thing, so it names the flaw, never the virtue.

SOURCE  
Guidelines · 5.3 Positive and Negative Distribution  
Guidelines · 5.2 Rubric Weights

SEE RED SHELL EXAMPLES 6 AND 7

## Every ask graded, and nothing nobody asked for

Walk the prompt and tick off every ask: anything you can tick needs a criterion. In the other direction, every criterion traces back to something the prompt actually asks.

**Why** A missing critical requirement and a criterion the prompt never called for both make the grade come out wrong: Major, and Major issues on more than 10% of the criteria fail the task.

SOURCE  
Guidelines · 5.7 Final Rubric Checklist  
Guidelines · 5.6 How the Client Defines Rubric Issue Severity

SEE RED SHELL EXAMPLES 8 AND 9 MOST OFTEN BROKEN

## Weight is difficulty, never importance

Price how hard a criterion is to satisfy, or for a negative, how tempting the failure is. Count the axes it exercises: source coordination, reasoning depth, modality, discovery effort. Three or more is +5, exactly two is +3, one or none is +1.

**Why** A requirement can be critical and still +1, because meeting it is mechanical. A weight that ignores the axes is Incorrect Weights, a Moderate issue.

SOURCE  
Guidelines · 5.2 Rubric Weights  
Guidelines · 5.6 How the Client Defines Rubric Issue Severity

SEE RED SHELL EXAMPLE 10

# A rubric that separates two runs

A rubric exists to tell a strong run from a weak one. Five standards keep it doing that, and a set can break any of them while still looking complete.

## At most 20% process criteria

At least 80% of the objective criteria grade completion. Tool Use, Agent Behavior, every Trajectory target and narration of the process are the 20% at most, and zero is preferred. Trajectory is for what only the reasoning path shows: an identification, a dependency order, a piece of restraint.

**Count the objective block** · and round down: 12 criteria allow 2, 42 allow 8

**The test** · could the deliverable be perfect and this criterion still fail? Then it is process

**Why** Whatever the deliverable itself shows is graded on the deliverable. Past the cap the set grades the route instead of the result, and the task fails automatically. Red Shell capped trajectory criteria at five.

SOURCE

Guidelines · 5.1 Rubric Fundamentals

Guidelines · 5.4 Category & Evaluation Target

QC spec · Rubric Criteria - 80/20 Outcome Split (Process Over Cap)

**SEE RED SHELL EXAMPLE 11**

## Model A has to fail at least 30%

A hard client requirement: the Model A trajectory shows genuine failures worth at least 30% of the final rubric score, with meaningful impact on task completion and the user experience. 50% or more is preferred.

**Trivial Task** · Model A scoring above 70% on the rubric fails the task

**Authentic failures** · never forced, never artificially scored to meet the threshold

**Why** The difficulty comes from the idea. If the model succeeds too easily, revisit the scenario's complexity; heavier weights on the same rows change nothing.

SOURCE

Guidelines · Hard Client Requirements for Every Task

QC spec · Rubric Criteria - All Criteria Scoring

RED SHELL ASKED FOR 50%

# Criteria a correct run can pass

A criterion has to pass a correct run, fail a wrong one, carry the right Category and Evaluation Target, and count once. If your own golden fails one, check the criterion first: it is usually the one that is broken.

## The golden passes the complete rubric

Your golden has to pass every objective criterion. Anything it fails is a broken criterion, not a broken golden. The usual cause is a rule only the Desired Outcome states: the agent only ever sees the prompt and the inputs.

**Why** A criterion a correct run cannot satisfy is Major. Check before you complete Leg B, while a cleanup turn can still fix the golden if the golden is what is wrong.

SOURCE

Guidelines · 5.7 Final Rubric Checklist  
Guidelines · 4. Valid vs. Invalid Model Failures

SEE RED SHELL EXAMPLES [12 ↗](#) AND [13 ↗](#)

## A criterion is as strict as its source

Grade only what the scenario settles objectively. A criterion accepts every valid implementation and rejects the invalid ones: no exact figure the source leaves open, no one answer where several are defensible, no single route where two sources carry the value.

**Why** Too specific rejects valid runs; too loose accepts wrong ones: Overfitting and Underfitting, a Moderate issue. If you are unsure which answer is right, the GTFA is not settled.

SOURCE

Guidelines · 5.1 Rubric Fundamentals  
Guidelines · 5.6 How the Client Defines Rubric Issue Severity

SEE RED SHELL EXAMPLES [14 ↗](#), [15 ↗](#) AND [16 ↗](#)

MOST OFTEN BROKEN

# Criteria a correct run can pass

A criterion has to pass a correct run, fail a wrong one, carry the right Category and Evaluation Target, and count once. If your own golden fails one, check the criterion first: it is usually the one that is broken.

## Category and Evaluation Target

The target is mechanical, set by where the grader looks. The category says what kind of check it is, and Agent Behavior always sits on Trajectory.

**Final Answer Artifact** · a file the agent created or modified

**State Change** · a record changed in the service, through snapshots.json

**User-Facing Message** · the final assistant message

**Trajectory** · the agent's reasoning path, counted in the 20% process cap

**Why** Both fields shape how the rubric data reaches the client. In the subjective block Task Completion is the only valid category: anything the prompt asked for is objective.

### SOURCE

Guidelines · 5.4 Category & Evaluation Target

Guidelines · 7.4 Categories and Evaluation Targets for Subjective Rubrics

SEE RED SHELL EXAMPLES [17](#) ^ AND [18](#) ^

## Each piece of evidence is graded once

Two criteria decided by the same evidence are one check counted twice: one swallowed by a broader one, a negative that mirrors a positive, or a subjective row restating an objective value.

**Why** A single slip then costs the run twice. A check in the wrong block is moved to the right one, never duplicated across both, and two value checks in the subjective block fail the task.

### SOURCE

Guidelines · 5.3 Positive and Negative Distribution

Guidelines · 5.6 How the Client Defines Rubric Issue Severity

QC spec · Rubric Criteria - Subjective Block Scope

SEE RED SHELL EXAMPLE [19](#) ^

# The subjective block

Ten or more criteria, each grading one visible property of the render rather than your golden's wording, and each naming its file when the task ships more than one. Check all three before you submit.

## Anchor each criterion to a named file

Each subjective criterion points to one identifiable element and one visible property, and it has to be rateable from itself and its target alone. When the task ships more than one file, the filename is part of telling the reviewer where to look.

**Why** The target only says Final Answer Artifact. With two deliverables, "the table" identifies nothing, and two graders can rate the same run against different files.

SOURCE  
Guidelines · 5.1 Rubric Fundamentals  
Guidelines · 7.1 Writing Subjective Rubrics

SEE RED SHELL EXAMPLE [20](#) ↗

## Ten or more criteria

The subjective block expects 10 or more criteria, from a deliverable rich enough to justify them, never padded to hit the count.

**Cover the render** · layout, legibility, hierarchy, structure, consistency, register  
**Weight is impact on the user** · not how hard the property was; Red Shell's 30% cap is gone

**Why** Ten is a floor, not a target. A block clustered on one region leaves the rest of the render ungraded, and two runs presenting the same figures very differently score the same.

SOURCE  
Guidelines · 7.1 Writing Subjective Rubrics  
Guidelines · 7.2 Subjective Rubric Weights

SEE RED SHELL EXAMPLE [21](#) ↗

## Grade a property of the render

Subjective criteria grade the render side the prompt never asked for: layout, hierarchy, spacing, readability, structure, consistency. Name one observable property, never the wording one build happened to use.

**Why** Anything the prompt fixed is objective only. A criterion quoting your golden's headings rejects every valid page worded differently: that is Overfitting.

SOURCE  
Guidelines · 7.1 Writing Subjective Rubrics  
QC spec · Rubric Criteria - Subjective Block Scope

SEE RED SHELL EXAMPLE [22](#) ↗

# Closing the task

The golden is a deliverable, not a run you stopped. Check every value against its source, then run the close out before you submit.

## Every golden claim is grounded in the inputs

The golden is the ideal response, and every factual claim in it is grounded in the actual inputs: images opened, videos watched, PDFs read. That includes values no criterion grades.

**Why** The subjective block and the pairwise comparison are judged against the golden, so a wrong value in it is scored as correct. An inaccuracy or an ungrounded claim is a Minor Golden Defect.

SOURCE  
Guidelines · 6. Golden Solution - Leg B  
QC spec · Golden Solution - Artifact Completeness

SEE RED SHELL EXAMPLES [23](#) AND [24](#)

## The golden ships finished and complete

Every file the prompt names, spelled as the prompt spells it, and nothing describing how the work was made. Star the preferred run, and copy every literal in a criterion from its source.

**Fails the task** · a file the prompt names is missing  
**Costs points** · narration or a build script left with the golden; no preferred run marked

**Why** The golden ships artifacts only: a script or build note left with it shows how the answer was reached. A missing star costs points on its own.

SOURCE  
QC spec · Golden Solution - Artifact Completeness  
QC spec · Trajectory - Golden/Preferred Run Selection  
Guidelines · 5.1 Rubric Fundamentals

SEE RED SHELL EXAMPLES [25](#) AND [26](#)

# The checks, section by section

If you read one slide twice, read this one. One check per standard in this deck, and every one of them a real Red Shell task failed.

## 1. THE FOUNDATIONS EVERY TASK MUST GET RIGHT

- 1 The deliverable clears its floor on the Complexity Bar
- 2 The universe returns every record your rules point at

## 2. WHAT YOU MAY NOT TELL MODEL A OR MODEL B

- 1 Nothing you type gives a value, count, position or conclusion the inputs should supply
- 2 One prompt to Model A, no follow ups; Leg B runs it word for word
- 3 Every Leg B hint points at the area, never at the answer or the exact file

## 3. A RUBRIC THAT SEPARATES TWO RUNS

- 1 About a quarter negatives, each a trap the scenario really sets
- 2 Every ask has a criterion, and every criterion an ask
- 3 Each weight matches the axes its criterion exercises
- 4 Process criteria at most 20% of the objective block, ideally none
- 5 Model A genuinely fails at least 30% of the rubric

## 4. CRITERIA A CORRECT RUN CAN PASS

- 1 The golden passes every objective criterion, checked before you complete Leg B
- 2 Every criterion is as strict as its source, no more and no less
- 3 Agent Behavior sits on Trajectory; every subjective row is Task Completion
- 4 No two criteria are decided by the same evidence

## 5. THE SUBJECTIVE BLOCK

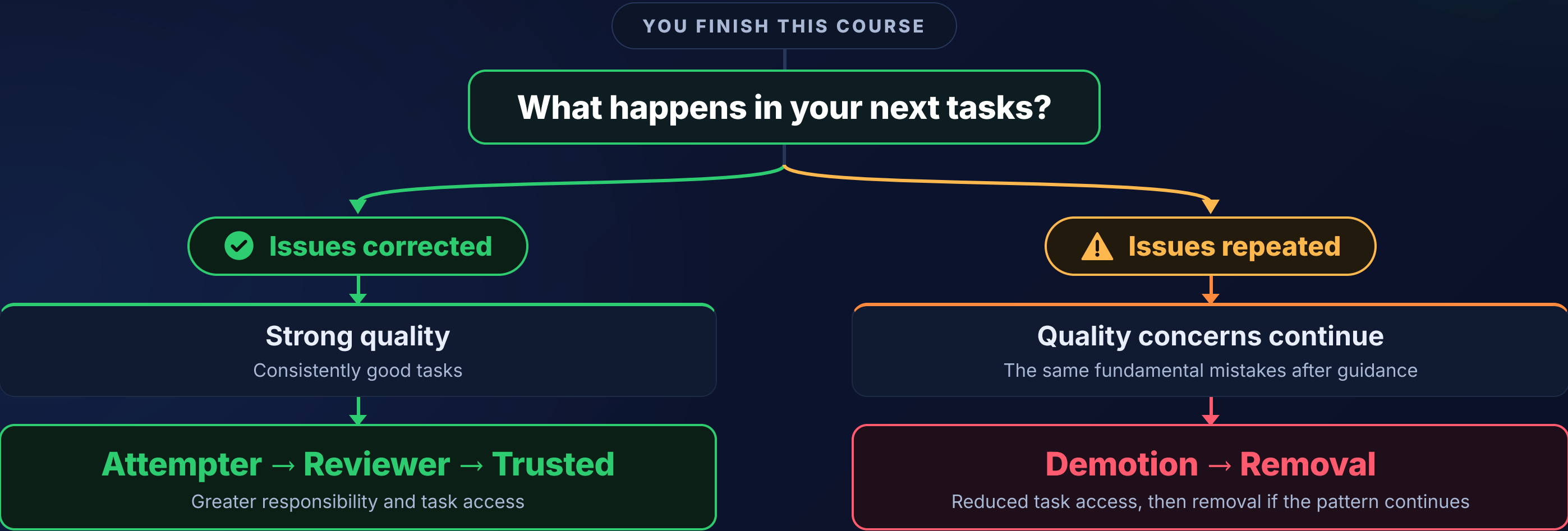
- 1 Every subjective criterion names the file it grades
- 2 Ten or more subjective criteria
- 3 Subjective criteria grade properties, never your own wording

## 6. CLOSING THE TASK

- 1 Every golden value matches its source, everywhere it appears
- 2 The golden holds every named file, spelled verbatim, and nothing else
- 3 Every literal copied from its source; the preferred run starred

# What happens from here is simple

You have seen the mistakes that kept costing tasks and project access in Red Shell. Nobody is expected to be perfect, but once an issue has been identified and explained, it is expected to be corrected, not repeated.



These are not hidden expectations anymore: the standards, the real examples and how to meet them are all in this course.

The goal is not to catch you making mistakes. It is to make sure the same preventable mistakes stop happening.

Thank you for your engagement and cooperation.